Übungsblatt 7: Performance und Pipelining

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Lohnt sich die Pipeline für den Sensorknoten? (10 Punkte)

Ein Sensorknoten überwacht den Füllstand eines Wassertanks; er begleitet Sie durch das ganze Blatt. Sein RV32I-Prozessor soll entweder als Single-Cycle-Prozessor oder mit der fünfstufigen Pipeline des Kurses gebaut werden. Es gelten die Lehrwerte der Vorlesung: IF 200 ps, ID 100 ps, EX 150 ps, MEM 250 ps, WB 100 ps und 20 ps Registeroverhead je Taktpfad. Die Pipeline ist anfangs leer und hat keine Zusatzzyklen. Gezählt wird vom ersten IF bis zum letzten WB.

1. Eine Auswerteroutine schließt $N = 16$ Instruktionen ab. Ergänzen Sie die Pipeline-Spalte. Die Single-Cycle-Spalte ist ein Beispiel. (4 P)

Größe	Single Cycle	Pipeline
Taktperiode T	$200 + 100 + 150 + 250 + 100 + 20 = 820$ ps	
Latenz eines einzelnen Befehls	820 ps	
Takte C	16	
CPI $= C/N$	1	
Laufzeit $t = C \cdot T$	13 120 ps	

2. Berechnen Sie den Speedup der Pipeline für diese Routine und den Grenzwert für sehr große N. Nennen Sie zwei Gründe, warum der Grenzwert nicht fünf ist. (2 P)
3. Der Hersteller bietet schnellere Datenspeicher an; alle anderen Stufen bleiben gleich. Ergänzen Sie die Tabelle für $N = 16$. Die erste Zeile ist ein Beispiel. (3 P)

MEM-Zeit	T Single Cycle	T Pipeline	begrenzende Stufe	Laufzeit Pipeline
250 ps	820 ps	270 ps	MEM	5400 ps
180 ps				
120 ps				

4. Lohnt sich für die Pipeline der Schritt von 180 ps auf 120 ps? Für welchen der beiden Prozessoren lohnt er sich? (1 P)

Musterlösung

Größe	Single Cycle	Pipeline
Taktperiode T	820 ps	$250 + 20 = 270$ ps
Latenz eines einzelnen Befehls	820 ps	$5 \cdot 270 = 1350$ ps
Takte C	16	$16 + 4 = 20$
CPI $= C/N$	1	$20/16 = 1{,}25$
Laufzeit $t = C \cdot T$	13 120 ps	$20 \cdot 270 = 5400$ ps

1. Je 1 P für T, Latenz, C mit CPI und Laufzeit. Den Takt bestimmt die längste Stufe MEM. Die Pipeline verlängert die Latenz eines Befehls, schließt aber mehrere Befehle überlappt ab.
2. $S = 13\,120/5400 \approx 2{,}43$ (1 P). Grenzwert $820/270 \approx 3{,}04$. Die Stufen sind ungleich lang, kürzere Stufen warten auf MEM; außerdem fällt der Registeroverhead in jedem Takt an (1 P).

MEM-Zeit	T Single Cycle	T Pipeline	begrenzende Stufe	Laufzeit Pipeline
180 ps	750 ps	220 ps	IF	4400 ps
120 ps	690 ps	220 ps	IF	4400 ps

3. Je Zeile 1,5 P. Laufzeit der Pipeline jeweils $20 \cdot 220 = 4400$ ps.
4. Nein: Ab 180 ps begrenzt IF mit 200 ps den Takt, die Pipeline bleibt bei 220 ps. Der Single-Cycle-Prozessor profitiert weiter, weil bei ihm die Summe aller Stufen zählt: 12 000 ps statt 11 040 ps für die Routine (1 P).

Aufgabe 2: Ergebnisse weiterleiten (10 Punkte)

Der Sensorknoten hat den Rohwert t0 = 97 erhalten. Er zieht den Nullpunkt-Offset s1 = 40 ab, korrigiert die Einbaulage um −2, addiert den Füllstand zur bisherigen Summe a0 = 120 und bildet die Abweichung vom Sollstand a2 = 60. Alle Anfangswerte stehen vor I1 in der Registerbank bereit. Die Pipeline leitet Ergebnisse aus EX/MEM und MEM/WB an die EX-Stufe weiter (Forwarding).

```
sub  t1, t0, s1      # I1  Füllstand = Rohwert - Offset
addi t1, t1, -2      # I2  Korrektur der Einbaulage
add  a0, a0, t1      # I3  Summe erhöhen
sub  t2, t1, a2      # I4  Abweichung vom Sollstand
```

1. Ergänzen Sie das Stufenraster. Die Zeile von I1 ist ein Beispiel. (3 P)

Befehl	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2								
I3								
I4								

2. Ergänzen Sie für jeden Registeroperanden in EX die Quelle und den Wert. Quelle ist die Registerbank (Wert in ID gelesen), EX/MEM oder MEM/WB. Die erste Zeile ist ein Beispiel. Tipp: Passen zwei Produzenten, braucht der Leser den Wert des jüngeren. (4 P)

Befehl	EX in Takt	Operand	Quelle	Wert
I2	4	t1	EX/MEM, von I1	57
I3		t1		
I3		a0		
I4		t1		
I4		a2		

3. Geben Sie t1, a0 und t2 nach Abschluss von I4 an. (1 P)
4. Nach einer Neujustierung entfällt die Korrektur. An die Stelle von I2 tritt der unabhängige Messwertzähler `addi a1, a1, 1`; I4 braucht nun den Wert 57 von I1. In welchem Takt steht I1 in WB, in welchem I4 in ID? Liefert in Takt 6 noch EX/MEM oder MEM/WB den Wert 57? Wie behandelt das Kursmodell diesen Fall, und wie viele Takte braucht die Folge? Tipp: Im Kursmodell ist ein in WB geschriebener Wert erst im Folgetakt in ID lesbar. (2 P)

Musterlösung

Befehl	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

1. Je Zeile 1 P. Es gibt keinen Wartezyklus, die Folge braucht $4 + 4 = 8$ Takte.

Befehl	EX in Takt	Operand	Quelle	Wert
I3	5	t1	EX/MEM, von I2	55
I3	5	a0	Registerbank	120
I4	6	t1	MEM/WB, von I2	55
I4	6	a2	Registerbank	60

2. Je Zeile 1 P. In Takt 5 passt für t1 auch I1 in MEM/WB mit 57; der jüngere Produzent I2 gewinnt, weil er in Programmreihenfolge zuletzt schreibt.
3. t1 = 55, a0 = 175, t2 = −5 (1 P).
4. I1 steht in Takt 5 in WB, I4 im selben Takt in ID und liest dort noch den alten Inhalt von t1. In Takt 6 hat I1 die Pipeline verlassen: EX/MEM hält I3, MEM/WB den Zähler; keines liefert 57 (1 P). Das Kursmodell hält I4 einen Takt in ID, eine Bubble läuft nach EX, und I4 liest in Takt 6 erneut. EX folgt in Takt 7, die Folge braucht 9 Takte. Ergebnisse t1 = 57, a0 = 177, t2 = −3; der Zähler a1 steigt um eins (1 P). Eine Registerbank, die im selben Takt zuerst schreibt und dann liest, bräuchte keinen Halt.

Aufgabe 3: Den Rohwert aus dem Speicher laden (10 Punkte)

Jetzt lädt der Sensorknoten den Rohwert selbst aus dem Messpuffer. s0 = `0x1000` zeigt auf das Wort 97, an `0x1004` liegt der nächste Rohwert 103. Außerdem gilt s1 = 40, a0 = 120 und a1 = 4 (Messwertzähler). Ein Load liefert sein Wort erst nach MEM. Braucht der unmittelbar folgende Befehl es schon in EX, entsteht ein Load-Use-Stall: PC und IF/ID halten, der Verbraucher bleibt in ID, nach ID/EX läuft eine wirkungslose Bubble.

```
lw   t0, 0(s0)      # I1  Rohwert laden
sub  t1, t0, s1     # I2  Füllstand = Rohwert - Offset
add  a0, a0, t1     # I3  Summe erhöhen
addi a1, a1, 1      # I4  Messwertzähler erhöhen
```

1. Ergänzen Sie das Stufenraster. Markieren Sie gehaltene Stufen mit einem Stern, zum Beispiel ID*, und tragen Sie die Bubble in ihre eigene Zeile ein. (4 P)

Eintrag	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2									
I3									
I4									
Bubble									

2. Am Ende von Takt 3 übernimmt EX/MEM das Ergebnis der EX-Stufe des Loads. Welcher Wert ist das? Warum darf I2 ihn nicht verwenden? (1 P)
3. Geben Sie Quelle und Wert von t0 für I2 und von t1 für I3 in deren EX-Takt an. (2 P)
4. Geben Sie t0, t1, a0 und a1 nach Abschluss an. (1 P)
5. Bestimmen Sie N, C und den endlichen CPI. Zählt die Bubble zu N? Wie lange dauert der Kern bei 270 ps? (2 P)

Musterlösung

Eintrag	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	ID*	EX	MEM	WB		
I3			IF	IF*	ID	EX	MEM	WB	
I4					IF	ID	EX	MEM	WB
Bubble				EX	MEM	WB			

1. Je Zeile 1 P. Der Load läuft weiter, I2 und I3 warten einen Takt.
2. Die Adresse $s0 + 0 = \texttt{0x1000}$ (4096). Das Wort 97 liest erst MEM in Takt 4; die Adresse ist nicht der Rohwert (1 P).
3. I2 in EX5: t0 = 97 aus MEM/WB. I3 in EX6: t1 = 57 aus EX/MEM. Je 1 P.
4. t0 = 97, t1 = 57, a0 = 177, a1 = 5 (1 P).
5. $N = 4$, $C = 9$, CPI $= 9/4 = 2{,}25$; die Bubble ist kein Programmbefehl und zählt nicht zu N (1 P). $t = 9 \cdot 270 = 2430$ ps (1 P).

Aufgabe 4: Umordnen statt warten (10 Punkte)

Der Wartetakt aus Aufgabe 3 soll verschwinden, ohne die Arbeit des Sensorknotens zu verändern. Der Anfangszustand ist derselbe wie in Aufgabe 3.

1. Ordnen Sie die vier Befehle aus Aufgabe 3 so um, dass kein Load-Use-Stall entsteht und alle Endwerte gleich bleiben. Begründen Sie, warum der verschobene Befehl unabhängig ist. (2 P)
2. Ergänzen Sie das Stufenraster Ihrer Reihenfolge. Die erste Zeile ist vorgegeben. Wie viele Takte braucht der Kern jetzt? (2 P)

Befehl	1	2	3	4	5	6	7	8	9
`lw t0, 0(s0)`	IF	ID	EX	MEM	WB				

3. Vergleichen Sie die Laufzeiten vorher und nachher bei 270 ps und berechnen Sie den Speedup des Kerns. (2 P)
4. Ein Kollege schlägt stattdessen vor, in der Reihenfolge aus Aufgabe 3 den NOP `addi x0, x0, 0` zwischen `lw` und `sub` einzufügen. Bestimmen Sie N, C und CPI. Der CPI sinkt gegenüber Aufgabe 3. Ist das Programm dadurch schneller? (2 P)
5. Für den nächsten Messwert muss zusätzlich `addi s0, s0, 4` ausgeführt werden. Ausgehend von der Reihenfolge aus Aufgabe 3 setzt ein Kollege ihn ganz an den Anfang, vor den Load. Welche Werte erhalten t0, t1 und a0? Wo darf der Befehl stattdessen stehen? (2 P)

Musterlösung

1. Reihenfolge `lw`, `addi a1, a1, 1`, `sub`, `add` (1 P). Der Zähler liest und schreibt nur a1. Er verändert weder s0 noch t0, t1, a0 oder den Speicher, und kein anderer Befehl braucht a1 (1 P).

Befehl	1	2	3	4	5	6	7	8	9
`lw t0, 0(s0)`	IF	ID	EX	MEM	WB				
`addi a1, a1, 1`		IF	ID	EX	MEM	WB			
`sub t1, t0, s1`			IF	ID	EX	MEM	WB		
`add a0, a0, t1`				IF	ID	EX	MEM	WB	

2. Raster 1 P. Acht Takte; `sub` erhält t0 = 97 in EX5 aus MEM/WB, `add` erhält t1 = 57 in EX6 aus EX/MEM (1 P).
3. Vorher $9 \cdot 270 = 2430$ ps, nachher $8 \cdot 270 = 2160$ ps (1 P). Speedup $2430/2160 = 9/8 = 1{,}125$ (1 P).
4. $N = 5$, $C = 9$, CPI $= 9/5 = 1{,}8$ (1 P). Nein: $9 \cdot 270 = 2430$ ps wie mit Stall. Der NOP belegt nur den Platz der Bubble und zählt zu N; deshalb sinkt der CPI, die Laufzeit nicht (1 P).
5. Vor dem Load liest `lw` an `0x1004`: t0 = 103, t1 = 63, a0 = 183, also der falsche Messwert (1 P). Der Befehl muss hinter dem Load stehen, zum Beispiel am Ende oder direkt nach `lw`; dort füllt er sogar die Lücke wie der Zähler (1 P).

Aufgabe 5: Fehlersuche nach einem Umbau (10 Punkte)

Ein Team hat die Pipeline des Sensorknotens nachgebaut. Es testet mit den Programmen aus Aufgabe 2 und 3 bei denselben Anfangswerten.

Symptom 1. Nach dem Programm aus Aufgabe 3 zeigt die Registeransicht t0 = 97, aber t1 = 4056 und a0 = 4176.

1. Welche Werte müssten t1 und a0 haben? Rechnen Sie zurück: Welchen Wert hat I2 für t0 verwendet? Geben Sie ihn auch hexadezimal an. (2 P)
2. Woher stammt dieser Wert, und in welchem Pipeline-Register stand er, als I2 rechnete? Welche Einheit arbeitet offenbar nicht, und was hätte sie in Takt 4 tun müssen? (2 P)
3. Mit der umgeordneten Folge aus Aufgabe 4 liefert derselbe Aufbau a0 = 177. Ist der Fehler damit behoben? Begründen Sie. (2 P)

Symptom 2. Ein anderer Aufbau liefert mit dem Programm aus Aufgabe 2 (mit Korrektur I2) t1 = 55 und t2 = −5, aber a0 = 177 statt 175.

4. Welchen t1-Wert hat I3 verwendet, und aus welchem Pipeline-Register kam er? (2 P)
5. Welche Regel verletzt die Weiterleitungslogik? Warum ist t2 trotzdem richtig? (2 P)

Musterlösung

1. Richtig wären t1 = 57 und a0 = 177. Aus $t1 = x - 40 = 4056$ folgt $x = 4096$ = $\texttt{0x1000}$; Kontrolle $120 + 4056 = 4176$. Je 1 P.
2. $\texttt{0x1000}$ ist die Load-Adresse $s0 + 0$, das Ergebnis der EX-Stufe des Loads. Sie stand in Takt 4 in EX/MEM und wurde weitergeleitet (1 P). Die Load-Use-Erkennung fehlt: Sie hätte I2 in Takt 4 in ID halten, PC und IF/ID festhalten und eine Bubble nach ID/EX geben müssen. Dann hätte I2 in Takt 5 das Wort 97 aus MEM/WB erhalten (1 P). t0 selbst stimmt, weil der Load in WB das geladene Wort schreibt.
3. Nein (1 P). In der umgeordneten Folge steht $\texttt{sub}$ zwei Befehle hinter dem Load und erhält 97 aus MEM/WB; ein Load-Use-Fall tritt gar nicht auf. Der Fehler bleibt nur verdeckt und zeigt sich bei jeder unmittelbaren Nutzung eines geladenen Werts (1 P).
4. $177 - 120 = 57$: I3 hat den unkorrigierten Füllstand von I1 verwendet (1 P). Er kam in Takt 5 aus MEM/WB (1 P).
5. Passen zwei Produzenten, muss der jüngere in EX/MEM Vorrang haben, hier I2 mit 55 (1 P). Wenn I4 in Takt 6 rechnet, passt nur noch I2 in MEM/WB, I1 hat die Pipeline verlassen. Die falsche Rangfolge wirkt dort nicht (1 P).

Aufgabe 6: Ausbaustufe: Nur neue Messwerte verarbeiten (10 Punkte)

Der Sensor meldet in t3, ob ein neuer Messwert vorliegt (1) oder nicht (0). Ohne neuen Wert soll der Sensorknoten die Anzeige nicht beschreiben und den Messwertzähler a1 nicht erhöhen; der Durchlaufzähler a3 zählt jeden Durchlauf. Der Branch wird in EX entschieden, bis dahin wird sequenziell geholt. Bei genommenem Branch werden die zwei jüngeren Einträge verworfen (Flush). Anfang: t1 = 57 (neuer Füllstand), s2 = `0x1100` zeigt auf das Anzeigewort mit dem alten Füllstand 50, a1 = 5, a3 = 9. Alle Werte stehen bereit.

```
        beq  t3, x0, weiter   # I1  kein neuer Messwert?
        sw   t1, 0(s2)        # I2  Füllstand anzeigen
        addi a1, a1, 1        # I3  Messwertzähler
weiter: addi a3, a3, 1        # I4  Durchlaufzähler
```

1. Es gilt t3 = 0. Ergänzen Sie das Stufenraster bis zum WB von I4. Markieren Sie verworfene Einträge ab dem Takt, in dem sie ungültig sind, mit ×. (3 P)

Eintrag	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2								
I3								
I4								

2. Geben Sie Anzeigewort, a1 und a3 danach an. Was zeigte die Anzeige, wenn die Hardware bei genommenem Branch nur den Fetch umlenkte, aber nichts verwürfe? (2 P)
3. Jetzt gilt t3 = 1. Welche Befehle schließen ab, und wie viele Takte braucht die Folge? (1 P)
4. Ein Messtag schließt $N = 200$ Instruktionen ab, darunter 30 Branches, von denen 12 genommen werden. Dazu kommen $H = 10$ Wartezyklen; die Zusatzkosten überschneiden sich nicht. Bestimmen Sie C, CPI und die Laufzeit bei 270 ps. Tipp: $C = N + 4 + H + 2F$. (2 P)
5. Der Statusbranch wird über acht Durchläufe so entschieden wie in der Tabelle (T = genommen, kein neuer Wert). Vergleichen Sie die feste Annahme Not Taken mit einem Einbitprädiktor, der mit N startet und nach jedem Branch die tatsächliche Richtung speichert. Jede Fehlvorhersage kostet zwei Takte, die Zieladresse steht rechtzeitig bereit. Wie viele Fehlvorhersagen und Zusatztakte ergeben sich jeweils? (2 P)

Durchlauf	1	2	3	4	5	6	7	8
tatsächlich	T	T	T	N	T	T	T	N
Vorhersage Einbit	N							
Fehler Einbit	ja							

Musterlösung

Eintrag	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	×				
I3			IF	×				
I4				IF	ID	EX	MEM	WB

1. Je Zeile 1 P. Der Branch entscheidet in EX3; an der Grenze zu Takt 4 werden I2 und I3 ungültig, das Ziel I4 startet in IF4.
2. Anzeigewort 50, a1 = 5, a3 = 10 (1 P). Ohne Verwerfen schriebe `sw` in MEM von Takt 5 den Wert 57, und a1 würde 6. Die Anzeige meldete einen neuen Füllstand, obwohl keiner vorlag (1 P).
3. Kein Flush, alle vier Befehle schließen ab: Anzeigewort 57, a1 = 6, a3 = 10. $C = 4 + 4 = 8$, ebenso viele Takte wie mit Flush, aber für vier statt zwei Befehle (1 P).
4. $C = 200 + 4 + 10 + 2 \cdot 12 = 238$ (1 P). CPI $= 238/200 = 1{,}19$, $t = 238 \cdot 270 = 64\,260$ ps = 64,26 ns (1 P).

Durchlauf	1	2	3	4	5	6	7	8
tatsächlich	T	T	T	N	T	T	T	N
Vorhersage Einbit	N	T	T	T	N	T	T	T
Fehler Einbit	ja	nein	nein	ja	ja	nein	nein	ja

5. Einbit: vier Fehler, acht Zusatztakte (1 P). Not Taken liegt bei jedem T falsch: sechs Fehler, zwölf Zusatztakte. Der Einbitprädiktor spart hier vier Takte (1 P).

Aufgabe 7: Den Kern in Ripes beobachten

Das Programm führt den Kern aus Aufgabe 3 mit denselben Anfangswerten aus. `la` lädt die Adresse des Messpuffers `messwerte` nach s0, `li` setzt die übrigen Anfangswerte. Die Schleife bei `ende` hält das Programm an und gehört nicht zum Kern.

```
.data
.align 2
messwerte: .word 97, 103
.text
.globl main
main:
    la   s0, messwerte
    li   s1, 40
    li   a0, 120
    li   a1, 4
kern:
    lw   t0, 0(s0)
    sub  t1, t0, s1
    add  a0, a0, t1
    addi a1, a1, 1
ende:
    beq  x0, x0, ende
```

1. Öffnen Sie Ripes, installiert oder im Browser unter ripes.dk. Geben Sie das Programm im Tab *Editor* ein; Ripes übersetzt es automatisch. Wählen Sie über *Select Processor* den *RISC-V 5-Stage Processor*. Er enthält Forwarding und Hazard-Erkennung, anders als die Varianten *w/o Forwarding or Hazard Detection* und *w/o Hazard Detection*. Aktivieren Sie unter *ISA Exts.* keine Erweiterung. Protokollieren Sie Ripes-Version und Modell.
2. Klicken Sie *Reset* und dann einzeln *Clock*, bis `addi a1, a1, 1` seine WB-Stufe durchlaufen hat. Öffnen Sie *Show stage table* und lesen Sie ab, in welchen Zyklen `lw` in IF und `addi a1, a1, 1` in WB steht. Die Kerntakte sind die Differenz plus eins. Vergleichen Sie Kerntakte, gehaltene Stufen und die Endwerte von t0, t1, a0 und a1 mit Aufgabe 3.
3. Verschieben Sie `addi a1, a1, 1` direkt hinter den Load wie in Aufgabe 4. Sagen Sie Kerntakte und Endwerte voraus, bevor Sie neu starten, und prüfen Sie dann.
4. Stellen Sie die Reihenfolge aus Teil 2 wieder her und fügen Sie `addi s0, s0, 4` vor dem Load ein. Sagen Sie t0, t1 und a0 voraus und prüfen Sie.
5. Sonderfall aus Aufgabe 2, Teil 4: Nehmen Sie die Reihenfolge aus Teil 3 und fügen Sie `addi s0, s0, 4` direkt hinter `addi a1, a1, 1` ein. Zwischen Load und `sub` stehen nun zwei unabhängige Befehle. Wie viele Kerntakte sagt das Kursmodell voraus? Hält Ripes `sub` in ID? Notieren Sie, welche Registerbank-Variante Ihre Ripes-Version zeigt.

Stufen in Ripes ablesen. Die Ansicht *Instruction memory* im Tab *Processor* zeigt in der Spalte *Stage*, in welcher Stufe ein Befehl gerade ist; die Registerwerte stehen in der Ansicht *Registers*. *Show stage table* zeigt für jeden Zyklus, welcher Befehl in welcher Stufe war. Gehaltene Stufen stehen dort als „-“. Aufgezeichnet wird nur beim Takten mit *Clock*, nicht bei *Run*. In der Prozessoransicht erscheinen Stalls und Flushes über der betroffenen Stufe als rot markiertes `nop`. *Reverse* nimmt einen Takt zurück. Die Zyklenzahl unter *Statistics* enthält auch die Initialisierung vor `kern`; die Kerntakte zählen Sie deshalb in der Stufentabelle. Die tatsächliche Adresse von `messwerte` kann von `0x1000` abweichen, die Registerwerte ändern sich dadurch nicht.

Musterlösung

Erwartung nach dem Kursmodell; B ist die Adresse von `messwerte`.

- Teil 2: neun Kerntakte. `sub` hält einen Takt in ID, `add` einen Takt in IF. Endwerte t0 = 97, t1 = 57, a0 = 177, a1 = 5.
- Teil 3: acht Kerntakte ohne Halt, dieselben Endwerte.

- Teil 4: s0 = B + 4, t0 = 103, t1 = 63, a0 = 183, a1 = 5. Der Zeiger verschiebt die Load-Adresse; das Ergebnis ist falsch, obwohl die Pipeline alle Abhängigkeiten korrekt behandelt.
- Teil 5: Endwerte s0 = B + 4, t0 = 97, t1 = 57, a0 = 177, a1 = 5. Der Load steht in WB, während `sub` in ID liest. Das Kursmodell hält `sub` einen Takt in ID und braucht zehn Kerntakte für fünf Befehle.

Laut Ripes-Quellcode besitzt die Registerbank des 5-Stufen-Modells einen Lesebypass: Ein im selben Takt in WB geschriebener Wert ist in ID bereits lesbar. Dann entfällt in Teil 5 der Haltetakt, und Ripes zeigt neun Kerntakte bei denselben Endwerten. Im Ripes-Labor gilt die beobachtete Variante; sie gehört mit Version und Modell ins Protokoll. Die Zyklenzahl unter *Statistics* enthält die Initialisierung, in der `la` zu zwei Maschinenbefehlen expandieren kann, und ist deshalb keine Kernzählung.